

BATCH-SCAMPP: Scaling phylogenetic placement methods to place many sequences

Eleanor Wedell 

Department of Computer Science, University of Illinois Urbana-Champaign, USA

Chengze Shen 

Department of Computer Science, University of Illinois Urbana-Champaign, USA

Tandy Warnow 

Department of Computer Science, University of Illinois Urbana-Champaign, USA

Abstract

Phylogenetic placement, the problem of placing sequences into phylogenetic trees, has been limited either by the number of sequences placed in a single run or by the size of the placement tree. The most accurate scalable phylogenetic placement method with respect to the number of query sequences placed, EPA-ng (Barbera et al., 2019), has a runtime that scales sub-linearly to the number of query sequences. However, larger phylogenetic trees cause an increase in EPA-ng’s memory usage, limiting the method to placement trees of up to 10,000 sequences. Our recently designed SCAMPP (Wedell et al., 2021) framework has been shown to scale EPA-ng to larger placement trees of up to 200,000 sequences by building a subtree for the placement of each query sequence. The approach of SCAMPP does not take advantage of EPA-ng’s parallel efficiency since it only places a single query for each run of EPA-ng. Here we present BATCH-SCAMPP, a new technique that overcomes this barrier and enables EPA-ng and other phylogenetic placement methods to scale to ultra-large backbone trees and many query sequences. BATCH-SCAMPP is freely available at https://github.com/ewedell/BSCAMPP_code.

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Phylogenetic Placement, EPA-ng, Phylogenetics

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

1 Introduction

Phylogenetic placement is the problem of placing one or more query sequences into a phylogenetic “backbone” tree, which may be a maximum likelihood tree on a multiple sequence alignment for a single gene or a taxonomy with leaves labeled by sequences for a single gene [22]. The most accurate techniques for phylogenetic placement use likelihood-based analyses and can be computationally intensive when the backbone trees are large. This application occurs when updating existing gene trees with newly observed sequences. However, phylogenetic placement can also be applied in the “bulk” context, where many sequences are placed at the same time into the backbone tree. For example, it can be applied when seeking to taxonomically characterize shotgun sequencing reads generated for an environmental sample in metagenomic analysis. Thus, phylogenetic placement is a general tool with applications in both incremental tree building and taxon identification and abundance profiling in microbiome analysis [8, 16, 20, 22, 7, 4]. In these analyses, however, the number of sequences placed into the backbone tree can be very large, especially when attempting to place all the shotgun sequencing reads in the sample. One way to reduce the number of reads for placement is to restrict the input to only those reads that map to one of the specified marker genes (which are genes universally found but appear only once in a given genome). Despite the restriction to a subset of the input data, using marker genes leads to improved accuracy in abundance profiling [20]. However, these restrictions can still result in



© Eleanor Wedell, Chengze Shen, and Tandy Warnow;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:17

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

thousands to tens of thousands of reads being placed in a single run into backbone trees.

Another aspect impacting the runtime of phylogenetic placement is the size of the backbone tree. Since many studies have shown the improvement in accuracy for abundance profiling, phylogenetic tree estimation, etc., when placing into large backbone trees [22], the importance of using phylogenetic placement methods that can run on large datasets is well established. Moreover, not all phylogenetic placement methods can scale to large backbone trees (and many methods are computationally intensive on large trees) [27, 26, 2], which has led to the development of methods, such as APPLES-2 [1], which use distances to place into large trees. There are also methods for phylogenetic placement that are alignment-free, such as those methods that use k-mers [12, 5]. These methods are much faster and more scalable than likelihood-based analyses, such as pplacer [14] and EPA-ng [3]. That said, likelihood-based methods are more accurate than these alignment-free methods and the distance-based methods [27]. Therefore, both the number of query sequences (i.e., sequences that need to be placed into trees) and the size of the backbone tree represent challenges for phylogenetic placement methods, especially when seeking to use likelihood-based placement methods.

Previously we introduced the SCAMPP framework [27] to enable both pplacer and EPA-ng to perform phylogenetic placement into ultra-large backbone trees, and we demonstrated its utility on backbone trees with up to 200,000 sequences. By using maximum likelihood methods pplacer or EPA-ng within the SCAMPP framework, the resulting placements are more accurate than with APPLES-2, with the most notable accuracy improvement for fragmentary sequences, and are computationally similar for single query sequence placement [27]. However, SCAMPP was designed to incrementally update a large tree, one query sequence at a time, and was not optimized for the other uses of phylogenetic placement, where batch placement of many sequencing reads is required. Furthermore, APPLES-2 has been improved since we designed SCAMPP!

EPA-ng is optimized for scaling the number of query sequences and is capable of placing millions of sequences into phylogenetic trees of up to a few thousand sequences [3]. EPA-ng performs this batch placement very efficiently, achieving sublinear runtime in the number of query sequences (see Figure 2 from [1]). However, other studies have shown that EPA-ng has a large memory requirement that depends on the size of the backbone tree; hence, the backbone trees used with EPA-ng are typically limited to a few thousand sequences. Thus, APPLES-2 has been the most scalable method capable of fast placement of many query sequences into ultra-large backbone trees [1].

Here we introduce BATCH-SCAMPP, a technique that improves scalability in both dimensions: the number of query sequences being placed into the backbone tree and the size of the backbone tree. BATCH-SCAMPP is based on SCAMPP, our prior method for scaling phylogenetic placement methods to large backbone trees. However, by its design, SCAMPP is not suitable for batch placement, so a significant modification to its design was required. Specifically, SCAMPP operates one query sequence at a time. Given the query sequence, SCAMPP finds and extracts a single (small) placement subtree out of the large backbone tree and then uses the selected phylogenetic placement method to place the query sequence into that placement subtree. Then, by superimposing the placement tree into the backbone tree and considering branch lengths, SCAMPP can identify the correct location within the original backbone tree to add the query sequence. While this approach enables phylogenetic placement methods to scale to ultra-large backbone trees, the fact that a different subtree is selected for each query sequence makes this approach ineffective for BATCH placement.

Our BATCH-SCAMPP method operates by allowing the input set of query sequences

to suggest and then vote on placement subtrees, thus enabling many query sequences to select the same placement subtree. We pair BATCH-SCAMPP with EPA-ng to explore the capability of this approach for scaling to many query sequences. We show that this combination of techniques (which we call BSCAMPP+EPA-ng, or BSCAMPP(e)) not only provides high accuracy and scalability to large backbone trees, matching that of SCAMPP, but also achieves the goal of scaling sublinearly in the number of query sequences. Moreover, it is much more scalable than EPA-ng and faster than SCAMPP-EPA-ng: when placing 10,000 sequences into a backbone tree of 50,000 leaves, **EPA-ng is unable to run due to memory issues, SCAMPP requires 1421 minutes, and BSCAMPP(e) places all sequences in 7 minutes (all given the same computational resources, see Table 3).**

The rest of the paper is organized as follows. We describe the overall divide-and-conquer approach of the BSCAMPP algorithm in Section 3. Since this approach requires algorithmic parameters to be set, we select the algorithmic parameters using a collection of simulated datasets (our training datasets) and then evaluate additional testing datasets. The design of this experimental study is described in Section 4. The study results are shown in Section 5. We provide a discussion of results in Section 6, and we finish with conclusions in Section 7.

2 Understanding EPA-ng accuracy

The goal of this experiment was to understand the impact on EPA-ng placement accuracy of the backbone tree size, when placing short sequences (the metagenomics context, see Supplementary Materials for results when placing also full-length sequences). We extracted a subset of the RNASim training dataset (see Section 4.4) for this experiment. The subset is a clade with 10,200 leaves of the training RNASim dataset, from which we randomly selected 200 leaves as queries. For the remaining 10,000 leaves, we randomly selected from 1000 to 10,000 leaves to form the backbone tree. For the queries, we made them into fragments ranging from 10%-length and 50%-length (results for 10%- and 25%-length are shown here, see Supplementary Materials for the full set of results). All queries were placed into the backbone trees using EPA-ng in one of two modes: “batch”, where we place all 200 queries all at once (a feature allowed by EPA-ng), and “one-by-one”, where we place each query independently by running EPA-ng on each query. We show delta error on various backbone tree sizes and placement strategies (batch vs. one-by-one) in Figure 1.

For both query sequence lengths, accuracy improves for one-by-one placement as the backbone size increases, indicating the beneficial impact of taxon sampling. Interestingly, this is not true for batch placement. For batch placement, phylogenetic placement accuracy is very much impacted by query length, but very fragmentary sequences are only placed well on small backbone trees (up to 2,000 leaves).

Since our interest is in placing short sequences (corresponding to read placement), we focus now on the results shown for queries of 10%-length. For these very short query sequences, we see a jump in delta error for “batch” from backbone tree size of 2000 to 5000 and up. Thus, high accuracy is only obtained when placing into smaller trees.

In sum, this experiment shows that EPA-ng performs very differently between the batch mode (the case where it is able to obtain a runtime advantage, according to prior studies) and the one-by-one mode. When the backbone tree is small enough then batch mode can be very accurate, but on larger backbone trees the one-by-one mode is more accurate for placing fragmentary sequences. The key lesson is that if we wish to obtain good accuracy in placing fragmentary sequences, we need to either use EPA-ng in one-by-one mode or keep the backbone tree at most 2000 sequences.

23:4 BATCH-SCAMPP: phylogenetic placement of many sequences

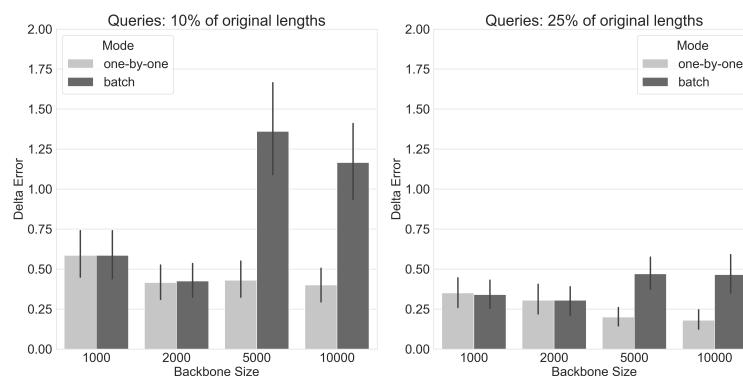


Figure 1 Mean delta error of EPA-ng when placing 200 queries into a backbone tree with 1000, 2000, 5000, and 10000 leaves of the training RNASim dataset. From left to right: query sequences (for placement) are made into fragments of 10% and 25%. We show two strategies for using EPA-ng for placement: “batch” means that all queries are placed at once, while “one-by-one” means that each query sequence is placed individually.

139 3 BATCH-SCAMPP

140 BATCH-SCAMPP (which we will call BSCAMPP for short) was designed with the goal
 141 of developing a divide-and-conquer strategy that allowed EPA-ng to scale datasets with
 142 large backbone trees (however, this approach could be used with any phylogenetic placement
 143 method). We assume that we have an input backbone tree T and its associated multiple
 144 sequence alignment and a set Q of query sequences that we will place into the tree T .

145 At a high level, our approach has four stages: Stage 1: the sequences in Q vote for their
 146 top v placement subtrees, each of bounded size B (both parameters specified by the user);
 147 Stage 2: We construct a set $\mathcal{T}$ of placement subtrees, using the votes, and we assign each
 148 query in Q to one of the placement subtrees; Stage 3: We allow reassignments of query
 149 sequences to subtrees; and Stage 4: we run EPA-ng on the placement subtrees to add the
 150 assigned query sequences into the subtrees, and then use branch lengths to find appropriate
 151 positions in the backbone tree T .

152 This four-stage approach is an elaboration on the SCAMPP technique, except that in
 153 SCAMPP, each query sequence picks a single placement subtree, and it is completely feasible
 154 that there will be as many placement trees as there are query sequences. Such an approach
 155 would fail to take advantage of the beneficial speed-up in EPA-ng when placing a large
 156 number of query sequences into the same subtree.

157 In a collection of initial experiments (all on the same training data), we explored different
 158 ways of implementing the same general divide-and-conquer pipeline. These variants differ
 159 mainly in Stage 2, which uses the votes to determine the subtree to assign each query sequence.
 160 The comparison between these variants is in the supplementary materials; here we show the
 161 variant that had the best accuracy and speed. We refer to this as BATCH-SCAMPP or
 162 BSCAMPP. This approach has two algorithmic parameters (subtree size B and the number
 163 v of votes), which we then explore here to determine the impact on accuracy and runtime.
 164

Algorithm BSCAMPP(e)

The input to BSCAMPP(e) is a placement tree T , with edge lengths, and a set Q of query sequences. BSCAMPP(e) has four stages, as we now describe.

- **Stage 1 (Initialization):**

- We compute the Hamming distance between every query sequence q in Q and every leaf in the tree T . Every query sequence q votes for the v leaves (ordered by Hamming distance to the query sequence), thus identifying their closest leaf, $closest(q)$.
- We initialize $\mathcal{T}$ to the emptyset.

- **Stage 2 (Constructing $\mathcal{T}$, the set of subtrees, and initial assignment of query sequences):**

- While there are some query sequences not yet assigned to any subtree, DO:
 - * Choose the most voted leaf x in the tree to use as a seed sequence, build a subtree using a breadth-first search based on branch length, until B leaves are in the subtree; call this tree t_x . We refer to x as the seed for t_x . Add subtree t_x to the collection $\mathcal{T}$.
 - * For every query sequence q , if the leaf $closest(q)$ is in t_x , then assign q to t_x and remove the votes from query sequence q

- **Stage 3 (Allow reassignments of query sequences):**

- For every query sequence q and for every tree $t_x \in \mathcal{T}$,
 - * compute the path distance (taking branch lengths into account) from $closest(q)$ to x . Let y_q be the seed sequence that has the minimum distance to q . Reassign q to t_{y_q} if $y_q \neq closest(q)$.

- **Stage 4 (Perform Placement):**

- For every subtree $t_x \in \mathcal{T}$,
 - * Run EPA-ng on t_x and its assigned query sequences, to produce t'_x
 - * Use technique from SCAMPP to add the query sequences in t'_x to T (the backbone tree). This technique uses branch lengths in the placement and full backbone tree.
- Return a file containing all placements, with their requisite confidence score, distal length, placement edge number, etc.

Implementation details On top of EPA-ng's parallel implementation, BSCAMPP(e) is written in python with certain parts written using openMP in C++. Since the Hamming distance computation is a computationally intensive portion of the BSCAMPP(e) framework (requiring $\mathcal{O}(rql)$ for q queries of length l compared to r reference leaves), a parallel implementation using openMP allows for easier batch processing of queries.

4 Performance Study Design

4.1 Overview

We evaluate placement methods for use with short sequences, the application similar to what would be encountered in placing short reads into phylogenetic trees or taxonomies. Since many reads are characterized in each run, scalability to large numbers of reads is a relevant question. We use both simulated and biological datasets in this study, dividing the datasets into training data and testing data.

Our training data are simulated datasets with known true trees, and the testing data are both biological datasets (with estimated maximum likelihood trees) as well as simulated datasets. We report placement error using "delta error", a standard metric used in prior studies [19, 2] (see Section 4.7).

We ran the following experiments for this study.

- Experiment 0 provides a study of scalability for EPA-ng and pplacer as a function of the number of query sequences, and motivates the development of BSCAMPP(e).
- Experiment 1 is the design of BSCAMPP(e) (i.e., BSCAMPP used with EPA-ng) on training data.
- Experiment 2 compares BSCAMPP(e) to EPA-ng, SCAMPP(p), SCAMPP(e), and APPLES-2 on testing data.
- Experiment 3 evaluates BSCAMPP(e) with a large backbone tree (180,000 leaves) and varying the number of query sequences.

See Supplementary Materials for additional details.

4.2 Methods

We refer to BSCAMPP used with EPA-ng by BSCAMPP(e), and when used with pplacer by BSCAMPP(p). Similarly, we refer to SCAMPP (v2.1.1) used with EPA-ng by SCAMPP(e) and when used with pplacer by SCAMPP(p). In our analyses, we use pplacer used with taxtastic [9] for placement, since this improves accuracy and scalability according to [?, 1]. We also examine APPLES-2 (v2.0.11) and EPA-ng (v0.3.8).

Each phylogenetic placement method requires numeric parameters to be estimated for the backbone trees. All backbone tree numeric parameters (branch lengths, 4×4 substitution rate matrix, etc.) are re-estimated according to the specifications of the phylogenetic placement method: RAxML-ng (v1.0.3) [11] for EPA-ng, SCAMPP(e), and BSCAMPP(e), and FastTree-2 (v2.1.11) [21] under Minimum Evolution for APPLES-2.

4.3 Computational resources

All methods are given four hours to run with 64 GB of memory and 16 cores. For SCAMPP(e) and SCAMPP(p), when placement time was over four hours (which occurred in all experiments with 10,000 query sequences), the query sequences were split into 40 subsets of 250 sequences each. SCAMPP(e) and SCAMPP(p) were then run 40 times for each subset containing 250 query sequences.

4.4 Datasets

All datasets are from prior studies and are freely available in public repositories.

4.4.1 RNASim

We use samples from the RNASim dataset [18], which is a simulated dataset of 1,000,000 sequences that evolve down a model tree under a biophysical model to preserve RNA secondary structure. Subsets of the million-sequence simulated dataset were used in prior studies evaluating phylogenetic placement methods [2, 1, 27], and provide a substantial challenge due to the dataset size. For this study, we split this dataset into two subsets by taking the model tree and splitting it into two clades, with one having approximately 600,000 sequences and the other having approximately 400,000 sequences. This defines two sets of sequences, with the smaller one used for training (Experiments 0 and 1) and the larger one for testing (Experiments 2 and 3). We place into a maximum likelihood tree on the true alignment (estimated using FastTree2), using the true alignment.

4.4.2 nt78

We also use the nt78 datasets, which were simulated for FastTree-2 [21]; these contain 10 replicates, simulated with Rose [25], each with 78,132 sequences in a multiple sequence alignment and the simulated backbone tree. We place into a maximum likelihood tree on the true alignment (estimated using FastTree2), using the true alignment. This dataset is used in the training phase (Experiment 1).

4.4.3 16S.B.ALL

For biological dataset analysis, we use 16S.B.ALL, a large biological dataset with 27,643 sequences and a reference alignment based on structure from CRW [6]. 16S.B.ALL contains some duplicate sequences; these were removed before analysis, producing a dataset with 25,093 sequences. Of these, 5,093 sequences were randomly selected as query sequences and the remaining were made backbone sequences. A maximum likelihood tree for this dataset was computed on this reference alignment using RAxML [24] and is available at [17]; this serves as the “reference tree” for this dataset and was reported in [13]. When computing delta error, we used the 75% bootstrap tree (i.e., the best ML tree found by RAxML, but with branches with support below 75%) collapsed) from [13] (available at [17]) as the reference topology.

4.5 Fragmentary sequence generation

We generated fragments from the full-length sequences starting at a randomly selected location. The fragmentary sequences are a mean of 10% of the original ungapped sequence length with a standard deviation of 10 nucleotides.

4.6 Additional details about experiments

In Experiment 1, we use $\sim 400,000$ sequences from the RNASim dataset, from which we randomly select 50,000 sequences to define the backbone sequences, 10,000 sequences for query sequences, and the remaining 340,000 sequences are not used. Sites of the true alignment on the 60,000 sequences containing more than 95% gaps are removed. To define the backbone tree on the set of sequences, we run FastTree-2 under the GTRGAMMA model on the true alignment.

In Experiment 0, we used a subset of the sequences in Experiment 1. Additionally, the 1,000-sequence backbone tree for Experiment 0 was built sub-sampling 1000 sequences from the 50,000-leaf tree. For the query sets, we randomly selected subsets of 10, 100, and 1000 queries from the 10,000 already chosen from Experiment 1.

For the nt78 datasets, we use the third replicate for the experiment. As the true alignment is not very gappy, we do not perform any masking. We pick 10,000 sequences at random for the query sequences and use the remaining 68,132 sequences as backbone sequences. We also define the backbone tree by running FastTree-2 on the true alignment of the backbone sequences.

4.7 Evaluation Criteria

We report placement error using average delta error, which is the increase in the number of false negatives (FN) produced by placing query sequences into the backbone tree. This requires the definition of the “true tree”, which is the model tree for the simulated data and

the published reference tree for the biological data. The delta error has been used in many previous studies [27, 1], and is the same as the node distance when the backbone tree is the true tree. The methods are also evaluated with respect to runtime and peak memory usage.

5 Results

5.1 Experiment 0: Motivating BSCAMPP+EPA-ng

The goal of this experiment was to understand the impact of varying the number of query sequences placed with EPA-ng and pplacer.

We show results on a 1000-leaf backbone tree for various numbers of query sequences (Figure 2). Note that EPA-ng takes less time to place 100 and 1000 sequences than it does to place 10 sequences, and more generally that its runtime grows sublinearly in the number of query sequences. Furthermore, neither its peak memory usage nor the delta error notably increase when placing more queries. In contrast, pplacer’s runtime grows much more quickly than EPA-ng’s.

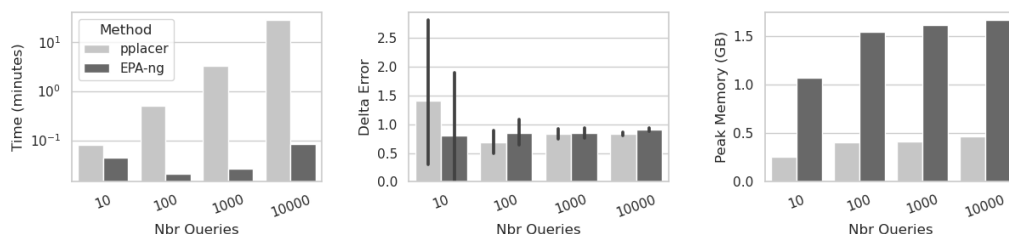


Figure 2 Experiment 0: Maximum-likelihood method comparison on training datasets on a 1000-leaf RNASim backbone tree. Runtime (left), Mean delta error (center), and peak memory usage in GB (right) are shown for placement of 10, 100, 1000, and 10,000 query sequences on an RNASim backbone tree with 1,000 leaves. We show results for pplacer and EPA-ng (both methods that can be used within the SCAMPP and BSCAMPP frameworks).

Furthermore, previous work has shown that EPA-ng can fail to run on larger backbone tree sizes once the subtree size reaches over 50,000 sequences. For example, [27] showed that EPA-ng is unable to place a single sequence in the tree on the nt78 dataset (with approximately 78,000 leaves) due to out-of-memory issues, whereas EPA-ng-SCAMPP (here referred to as SCAMPP(e)) is able to return a placement with low error. While the SCAMPP framework can place a sequence onto a large backbone tree, it was designed for placement of a single sequence, and will build a separate subtree for every query sequence. This will result in running EPA-ng for only one query sequence at a time, and thus the SCAMPP(e) cannot take advantage of EPA-ng’s sub-linear runtime scaling with respect to the number of query sequences.

This motivates the potential time saving utility of using EPA-ng to place multiple sequences in a single run within BSCAMPP.

5.2 Experiment 1: BSCAMPP(e) design

In order to understand how to set the size of the subtree and the number of votes for BSCAMPP, we performed a number of experiments on the two training data sets. We varied the subtree parameter setting from 1000, 2000, 3000, 5000 and 10,000 leaves. For each subtree size, we ran BSCAMPP(e) with 5 and 25 votes per query sequence.

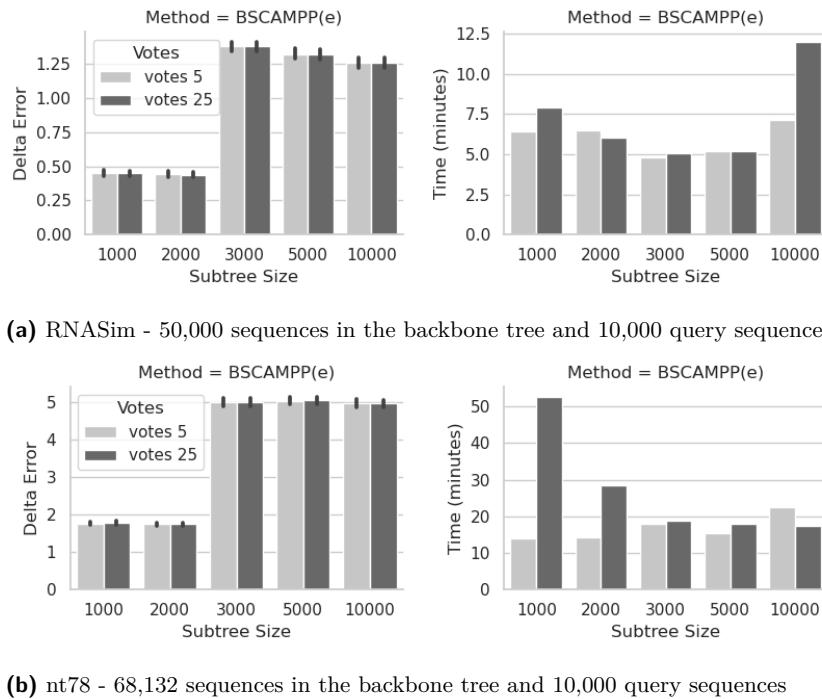


Figure 3 Experiment 1 - Subtree Size and Vote settings for BSCAMPP(e) on training datasets. Mean Delta Error plus standard error (left) and total runtime (right) for placement of all 10,000 fragmentary query sequences. We show placement time and error for BSCAMPP(e) varying parameter v (the number of votes per query) and the parameter B (the size of the subtree). The fragmentary sequences are a mean of 10% of the original ungapped sequence length with a standard deviation of 10 nucleotides.

We note a few trends with respect to the optimal subtree size. Most notably, Figure 3 shows that when the subtree size increases over 2,000 leaves, fragmentary sequence placement with BSCAMPP(e) has over twice the error than it does for 1,000- and 2,000-leaf subtrees. The number of votes does not show a large change in delta error for either dataset.

The runtime trends are dataset dependent: on the RNASim dataset, the placements with all of the subtree sizes and votes were able to complete in under 15 minutes, whereas the nt78 dataset required over 50 minutes for some settings. On the nt78 dataset, where the runtime was more affected by parameter settings, BSCAMPP(e) is the fastest at 14.1 minutes for 5 votes with a subtree size of 1000. For both subtree sizes where the delta error is low, using 5 votes for parameter v results in faster or similar runtimes. Based on these trends a subtree size of 2,000 and 5 votes are chosen as defaults for parameters B and v , respectively.

Using the chosen parameters, we show BSCAMPP(e) compared to other methods on the training datasets in Figure 4. On both training datasets, BSCAMPP(e) is nearly as accurate as SCAMPP(e) and SCAMPP(p) while taking a fraction of the runtime. SCAMPP(p) is the most accurate method, but takes 27 and 23 hours to complete on RNASim and nt78, respectively. APPLES-2 is the fastest method across both datasets, but shows the highest error. EPA-ng is unable to run, with the failure due to out-of-memory errors. However, BSCAMPP(e) is able to place all 10,000 sequences accurately and fast, while still using only 3 GB of memory.

23:10 BATCH-SCAMPP: phylogenetic placement of many sequences

■ **Table 1** Training Data Results for BSCAMPP(e) Parameter Settings on RNASim (50,000 sequences in the backbone tree with 10,000 fragmentary query sequences).

	RNASim			
	Delta Error		Runtime (minutes)	
Subtree Size	5 Votes	25 Votes	5 Votes	25 Votes
1000	0.450	0.447	6.4	7.9
2000	0.442	0.437	6.5	6.0
3000	1.376	1.377	4.8	5.0
5000	1.321	1.317	5.2	5.2
10000	1.257	1.258	7.2	12.0

■ **Table 2** Training Data Results for BSCAMPP(e) Parameter Settings on nt78 (68,132 sequences in the backbone tree with 10,000 fragmentary query sequences).

	nt78			
	Delta Error		Runtime (minutes)	
Subtree Size	5 Votes	25 Votes	5 Votes	25 Votes
1000	1.764	1.772	14.1	52.7
2000	1.744	1.734	14.4	28.4
3000	4.998	4.991	17.9	18.7
5000	5.028	5.038	15.5	18.0
10000	4.957	4.950	22.6	17.4

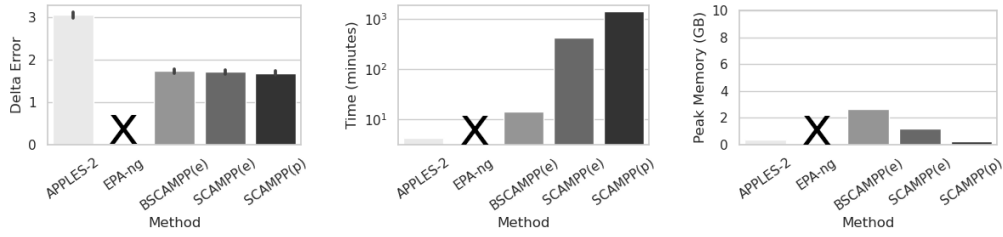
313 5.3 Experiment 2: Comparing BSCAMPP(e) to other methods

314 In this experiment, we compare our BSCAMPP(e) framework with the selected parameters
315 from Experiment 1 to SCAMPP [27], APPLES-2 [1], and EPA-ng [3] when placing fragmentary
316 sequences into a backbone tree.

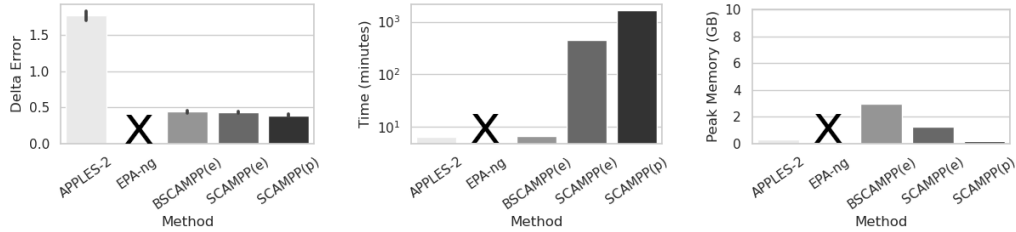
317 We show results for RNASim with 50,000 sequences in the backbone tree and 16S.B.ALL
318 with 20,000 sequences in the backbone tree in Figure 5. EPA-ng failed to run on the RNASim
319 dataset given 64 GB of memory, reporting an out-of-memory issue; however, EPA-ng was
320 able to place all sequences for the 16S.B.ALL dataset, within the memory limitations.
321 An examination of the memory usage shows that EPA-ng had a much higher memory
322 requirement (37 GB) on the 16S.B.ALL datasets, compared to the other methods (none of
323 which exceeded 3 GB). Thus, EPA-ng demonstrates challenges on these two datasets due to
324 memory requirements. All other methods succeeded in completing on these datasets within
325 the available time.

326 The methods differed substantially in terms of runtime to place all the query sequences
327 (10K for RNASim and 5093 for 16S.B.ALL). APPLES-2 never used more than 9 minutes on
328 either dataset, BSCAMPP(e) never used more than 8 minutes on either dataset, EPA-ng
329 used 17 minutes on 16S.B.ALL. In contrast, SCAMPP(e) used 214 and 466 minutes on
330 16S.B.ALL and RNASim, respectively, while SCAMPP(p) used 952 and 1421 minutes on
331 these two datasets. Thus, APPLES-2, BSCAMPP(e), and EPA-ng are very fast, completing
332 in just a few minutes on these datasets, and SCAMPP(e) and SCAMPP(p) are much slower.

333 The methods also differed in terms of placement accuracy. Specifically, APPLES-2 had
334 higher error than the likelihood-based methods for both datasets (delta error above 1.0 in
335 both cases), and EPA-ng had higher delta error than the other likelihood-based methods
336 on the one dataset on which it ran (16S.B.ALL). The comparison between BSCAMPP(e),



(a) nt78 (68,132 sequences in the backbone tree, placing 10,000 query sequences)



(b) RNASim (50,000 sequences in the backbone tree, placing 10,000 query sequences)

Figure 4 Experiment 1: Method Comparison on Training Datasets. Top: Mean delta error (left), runtime (center), and peak memory usage in GB (right) for placement of all 10,000 query sequences on the nt78 backbone trees with 68,132 leaves. Bottom: Mean delta error (left), runtime (center), and peak memory usage in GB (right) for placement of all 10,000 query sequences on the RNASim backbone trees with 50,000 leaves. We show placement time for BSCAMPP(e) for 5 votes using a subtree of 2000 leaves. SCAMPP(e) and SCAMPP(p) similarly use a subtree size of 2000 leaves. The results from APPLES-2 and SCAMPP are also included for runtime, peak memory usage, and delta error. EPA-ng was not shown because it was unable to run given 64 GB of memory and 16 cores on these datasets due to out-of-memory issues.

BSCAMPP(p), and SCAMPP(p) shows very close accuracy on both datasets (all close to 0.5), with perhaps a very slight edge to SCAMPP(p) on the RNASim dataset.

5.4 Experiment 3: Method comparison on ultra-large trees

Our final experiment is designed to verify that BSCAMPP(e) scales to both large query sets and ultra-large backbone tree sizes. Additionally, we check that BSCAMPP(e) is able to achieve sub-linear runtime scaling with respect to the number of query sequences. We use RNASim for this study, with a backbone tree having 180,000 leaves, and placing from 20 to 20,000 query sequences into the tree. We limited this study to BSCAMPP(e) and those methods that are able to place into the 50,000-leaf tree within 1 hour; this limited the study to only APPLES-2 and BSCAMPP(e). These analyses were run on the UIUC Campus Cluster, which is heterogeneous (i.e., some machines are older and hence slower than others). While both methods are given 16 cores and 64 GB, any given analysis may have access to very different computational resources.

Results in Figure 6 show that BSCAMPP(e) is able to place 20,000 sequences in just 25 minutes, whereas 20 sequences requires 1 minute. The runtime for BSCAMPP(e) increases by a factor of 6.7, 2.5, and 1.5 each time the number of queries increases tenfold. Similarly, APPLES-2 shows a sub-linear curve; the runtime increases by a factor of 3.6, 3.8, and then 6.9 each time the number of queries increases tenfold. APPLES-2 is faster for 20, 200, and 2,000 query sequences, but BSCAMPP(e) is faster at 20,000 query sequences. Moreover, both

23:12 BATCH-SCAMPP: phylogenetic placement of many sequences

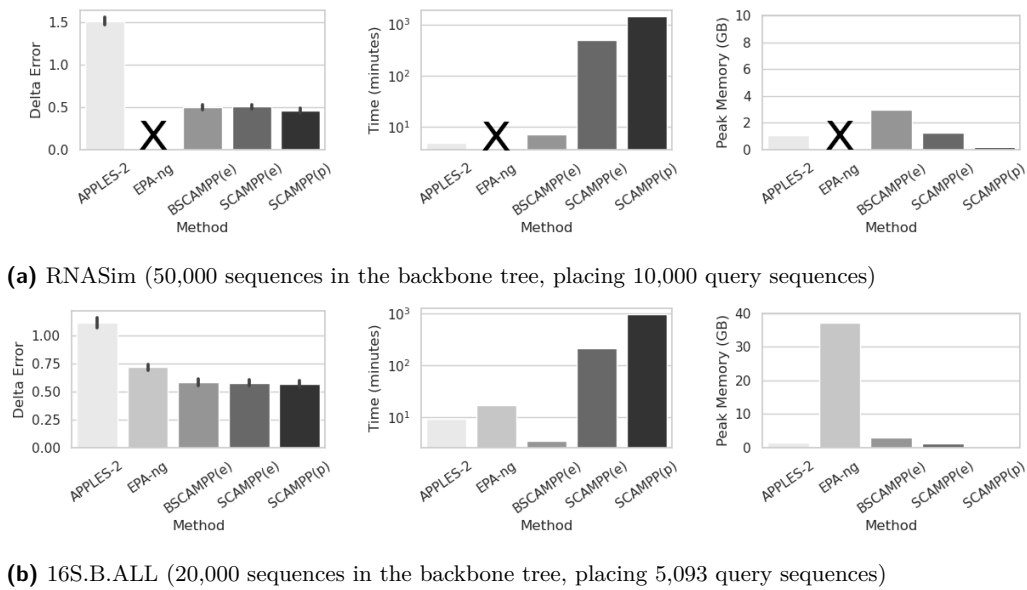


Figure 5 Experiment 2: Method Comparison on Testing Datasets - Top: Mean delta error (left), runtime (center), and peak memory usage in GB (right) for placement of all 10,000 query sequences on the RNASim backbone trees with 50,000 leaves. Bottom: Mean delta error (left), runtime (center), and peak memory usage in GB (right) for placement of all 5,093 query sequences on the 16S.B.ALL backbone trees with 20,000 leaves. We show placement time for BSCAMPP(e) for 5 votes with a subtree size of 2000. The results from APPLES-2 and EPA-ng are also included (an X indicates that EPA-ng was unable to run due to out-of-memory issues). SCAMPP(e) and SCAMPP(p) (using a taxtastic reference package) are both run for a subtree size of 2000.

methods are able to complete in under 1 hour, even when placing 20,000 query sequences into this large tree with 180,000 sequences. Finally, both methods use under 3 GB, well under the computational limit of 64 GB.

Results in Figure 6 also allow us to compare methods for accuracy, and see how changing the number of query sequences affects this. We see that BSCAMPP(e) delta error is not impacted by the number of query sequences, and when at least 200 sequences are considered the delta error for APPLES-2 is also not impacted; this makes sense. However, BSCAMPP(e) shows much lower delta error than APPLES-2 (consistent with trends in previous experiments).

6 Discussion

As seen in Table 3, EPA-ng is the least capable, as it had out-of-memory issues on all datasets we attempted except for 16S.B.ALL. APPLES-2 and BSCAMPP(e) were the two fastest, with APPLES-2 never using more than 47.1 minutes and BSCAMPP(e) never using more than 25.5 minutes. We also see that although SCAMPP(e) and SCAMPP(p) were able to run on all the datasets, they were by far the most computationally intensive of the tested methods. Furthermore, SCAMPP(e) used from 30 to 70 times as much time as BSCAMPP(e).

It is worth noting that our prior study evaluating SCAMPP [27] showed SCAMPP(p) being faster than APPLES-2 when placing a smaller number of query sequences. Thus, this study shows that the new version of APPLES-2 that we used in this study is significantly faster than the previous version we used in [27], and even so that we are able to be competitive

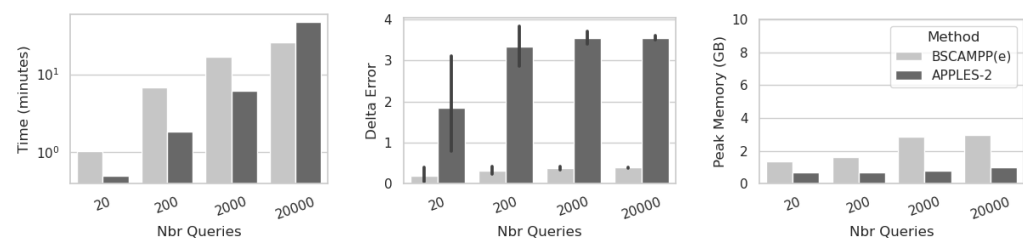


Figure 6 Experiment 3: Comparison of BSCAMPP(e) and APPLES2 given ultra-large backbone tree (180,000 sequences). We show total runtime (left) for placement of all query sequences, mean delta error (center), and peak memory usage (right). We run each method for 20, 200, 2,000, and 20,000 query sequences. Only APPLES-2 and BSCAMPP(e) are shown, since they were the only methods able to complete in under four hours for 20,000 queries.

Table 3 Runtime results across all datasets and backbone tree sizes for all methods. An “X” indicates that the analysis failed due to out-of-memory issues, and a “-” indicates the analysis was not attempted due to the expectation of excessive runtime (exceeding 500 minutes) or excessive memory requirements (exceeding 64 GB).

Method	Total Runtime (for all queries in minutes)				
	Training		Testing		
Dataset:	RNASim	nt78	RNASim		16S.B.ALL
Backbone Size:	50,000	68,132	50,000	180,000	20,000
Nbr Queries:	10,000	10,000	10,000	20,000	5,093
BSCAMPP(e)	6.5	14.4	7.2	25.5	3.5
SCAMPP(e)	457.4	429.2	466.0	-	214.6
SCAMPP(p)	1635.7	1426.4	1421.3	-	952.0
EPA-ng	X	X	X	-	17.4
APPLES-2	6.2	4.1	4.8	47.1	9.3

in terms of runtime when the number of query sequences is large enough.

We also note that BSCAMPP(e) is able to run on datasets where EPA-ng fails due to out-of-memory issues. Indeed, the only dataset in our study where EPA-ng was able to complete was the 16S.B.ALL dataset, and on this dataset, EPA-ng has higher error than BSCAMPP(e). The memory usage was never that high for any method that succeeded in running (the most was EPA-ng, at 37 GB), but as no method had access to more than 64 GB, it is possible that EPA-ng might have succeeded in running if it had access to a larger amount of memory.

Given these observations about scalability and computational performance, we now discuss accuracy. The first observation of this study is that BSCAMPP(e) (i.e., BATCH-SCAMPP used with EPA-ng) essentially matches the accuracy of the other likelihood-based placement methods (pplacer and EPA-ng, and their SCAMPP-boosted versions); furthermore, BSCAMPP(e) is much faster than the other likelihood-based placement methods when run on datasets with many query sequences. Our experiments consistently showed that all likelihood-based placement methods had very good accuracy with relatively minor differences between them (although EPA-ng was less accurate than the other likelihood-based methods) making computational performance the main distinguishing feature. Hence, the result that BSCAMPP(e) is much faster than the other likelihood-based methods is noteworthy. The second observation is that BSCAMPP(e) improves on the accuracy of APPLES-2, and even

improves on speed when the number of query sequences is large enough (at smaller numbers of query sequences, APPLES-2 is faster).

To understand why we see these trends, we consider what we learned about EPA-ng scalability on large backbone trees, both in terms of computational performance but also accuracy. Prior studies have suggested limits for EPA-ng to relatively small backbone trees due to computational reasons [27, 1, 2], but our Experiment 1 showed that EPA-ng had a jump in placement delta error as we increased the subtree size for the nt78 and RNASim datasets. Thus, our study suggests potentially that EPA-ng may have some numeric issues when placing into very large trees that result in increased placement error, a trend that has been previously observed for pplacer [27]. Thus, we would hypothesize that when placing into very large trees, even if EPA-ng manages to finish it may have reduced accuracy due to numeric issues, and that BSCAMPP(e) can have higher accuracy than EPA-ng since it restricts the placement subtrees for EPA-ng to be relatively small (at most 2000 sequences).

From a computational viewpoint, the main issue for EPA-ng is its large memory usage, which can explain why EPA-ng is limited to backbone trees of at most moderate size. Thus, by limiting the placement tree size to at most 2000 sequences, not only does BSCAMPP(e) succeed in maintaining good accuracy, but it also ensures good scalability to very large backbone trees.

We posit that our study suggests that BSCAMPP(e) may be able to scale to much larger numbers of query sequences while maintaining the sublinear runtime. The basis for this optimism is that EPA-ng has been shown to place millions of queries [3] into smaller trees of a few hundred sequences, yet we have limited this study to larger backbone trees with at most 20,000 query sequences. The other point is that Experiment 3 suggests that the full efficiency of BSCAMPP(e) has not yet been reached at 20,000 query sequences, and that it may continue to be able to place even larger numbers of query sequences with runtimes that grow sublinearly with that number even for backbone trees of this size.

BSCAMPP(e) allows for multiple queries to be placed into a subtree, using a new process for selecting subtrees and processing query sequences. We show in Experiment 3 that BSCAMPP(e) is able to achieve a sub-linear runtime scaling when increasing the number of queries; a similar sub-linear scaling to that of EPA-ng as shown in Experiment 0.

7 Conclusions

Phylogenetic placement into large backbone trees is fundamental to several bioinformatics problems, including microbiome analysis (e.g., taxonomic characterization of shotgun sequencing reads) and updating large phylogenetic trees. Yet, only a very small number of methods provide acceptable accuracy and scalability. Prior to this study, SCAMPP (used with maximum likelihood phylogenetic placement methods, such as pplacer or EPA-ng) had the best accuracy of all methods that could place into large backbone trees. However, by design SCAMPP did not ensure that the subproblems resulted in many query sequences being placed into the same subtree, which meant that it could not take advantage of EPA-ng's ability to place many query sequences with a runtime that is sublinear in the number of query sequences. By redesigning the divide-and-conquer strategy, we were able to ensure that many query sequences went to the same subproblems, and thus were able to take advantage of this feature of EPA-ng. Our redesign, which we call BATCH-SCAMPP (or BSCAMPP for short) matches the accuracy of the SCAMPP and scalability to large backbone trees, and when used with EPA-ng (i.e., BSCAMPP(e)) achieves sublinear runtime in the number of query sequences. Moreover, BSCAMPP is extremely fast, nearly as fast as APPLES-2 for

placing a moderate number of query sequences into a large backbone alignment, and slightly faster when placing large numbers of query sequences into very large backbone trees. Thus, BSCAMPP is the first likelihood-based phylogenetic placement method that is both highly accurate (due to its use of likelihood) and scalable in both dimensions – the number of query sequences and the size of the backbone tree (due to its use of divide-and-conquer and the reliance on EPA-ng for placement of many query sequences into small subtrees).

This study leaves several directions for future research. A possible improvement for the runtime can be explored by implementing parallel processing of subtrees (i.e., running instances of EPA-ng in parallel for different query-subtree sets). This might be particularly helpful in cases with few queries per subtree. Future work should include an analysis of the runtime and memory usage impacts of running EPA-ng concurrently on multiple compute nodes, in addition to running multiple instances of EPA-ng using fewer threads on a single compute node.

There are applications of phylogenetic placement methods that could be improved through the use of larger backbone tree sizes and many query sequences, now enabled with better accuracy through BSCAMPP. One such application is that of de novo phylogenetic tree building. Studies have shown that scalable phylogeny estimation methods have suffered in the presence of sequence length heterogeneity, and phylogenetic placement may provide a more accurate alternative [10]. For example, the divide-and-conquer tree estimation pipeline GTM (Guide Tree Merger) [23] could benefit by using BSCAMPP. BSCAMPP can facilitate the initial tree decomposition of GTM for better placement of shorter, fragmentary sequences into an initial tree containing the longer full-length sequences, potentially leading to better final tree estimation.

Another application of using more scalable backbone trees with many query sequences would be for methods of taxon identification and abundance profiling that use phylogenetic placement, such as TIPP [20] and its improved version TIPP2 [22], which have used pplacer [15]. As shown in [22], accuracy improves as the backbone trees increase in size, suggesting that the use of BSCAMPP(e) on large trees might lead to increased accuracy and reduced runtime compared to the current use of pplacer and smaller backbone trees.

8 Data availability

All datasets used in this study are from prior publications.

9 Acknowledgments

The authors acknowledge the financial support of the Department of Computer Science. EW was supported by a Siebel Scholars scholarship, a SURGE Fellowship, and a Wing Kai Cheng Fellowship.

476 — References —

- 477 1 Metin Balaban, Yueyu Jiang, Daniel Roush, Qiyun Zhu, and Siavash Mirarab. Fast and
478 accurate distance-based phylogenetic placement using divide and conquer. *Molecular Ecology*
479 *Resources*, 22(3):1213–1227, 2022.
- 480 2 Metin Balaban, Shahab Sarmashghi, and Siavash Mirarab. APPLES: scalable distance-based
481 phylogenetic placement with or without alignments. *Systematic Biology*, 69(3):566–578, 2020.
- 482 3 Pierre Barbera, Alexey M Kozlov, Lucas Czech, Benoit Morel, Diego Darriba, Tomáš Flouri,
483 and Alexandros Stamatakis. EPA-ng: massively parallel evolutionary placement of genetic
484 sequences. *Systematic Biology*, 68(2):365–369, 2019.
- 485 4 Holly M Bik, Dorota L Porazinska, Simon Creer, J Gregory Caporaso, Rob Knight, and
486 W Kelley Thomas. Sequencing our way towards understanding global eukaryotic biodiversity.
487 *Trends in Ecology & Evolution*, 27(4):233–243, 2012.
- 488 5 Matthias Blanke and Burkhard Morgenstern. App-SpaM: phylogenetic placement of
489 short reads without sequence alignment. *Bioinformatics Advances*, 1(1), 10 2021.
490 vbab027. arXiv:[https://academic.oup.com/bioinformaticsadvances/article-pdf/1/1/](https://academic.oup.com/bioinformaticsadvances/article-pdf/1/1/vbab027/41191909/vbab027.pdf)
491 [vbab027/41191909/vbab027.pdf](https://academic.oup.com/bioinformaticsadvances/article-pdf/1/1/vbab027/41191909/vbab027.pdf), doi:10.1093/bioadv/vbab027.
- 492 6 Jamie J Cannone, Sankar Subramanian, Murray N Schnare, James R Collett, Lisa M D’Souza,
493 Yushi Du, Brian Feng, Nan Lin, Lakshmi V Madabusi, Kirsten M Müller, Nupur Pande, Zhidi
494 Shang, Nan Yu, and Robin R Gutell. The comparative RNA web (CRW) site: an online
495 database of comparative sequence and structure information for ribosomal, intron, and other
496 RNAs. *BMC Bioinf*, 3(1):2, 2002.
- 497 7 Pierre-Alain Chaumeil, Aaron J Mussig, Philip Hugenholtz, and Donovan H Parks. GTDB-
498 Tk: a toolkit to classify genomes with the Genome Taxonomy Database. *Bioinformatics*,
499 36(6):1925–1927, 2020.
- 500 8 Sean Conlan, Heidi H Kong, and Julia A Segre. Species-level analysis of DNA sequence data
501 from the NIH Human Microbiome Project. *PloS one*, 7(10):e47075, 2012.
- 502 9 FHCRC. Taxtastic software, version 9.9.2, 2022. <https://github.com/fhcrc/taxtastic>, last
503 accessed Feb 20, 2022, distributed by Fred Hutchison Computational Biology.
- 504 10 Stefan Janssen, Daniel McDonald, Antonio Gonzalez, Jose A Navas-Molina, Lingjing Jiang,
505 Zhenjiang Zech Xu, Kevin Winker, Deborah M Kado, Eric Orwoll, Mark Manary, et al.
506 Phylogenetic placement of exact amplicon sequences improves associations with clinical
507 information. *Msystems*, 3(3):e00021–18, 2018.
- 508 11 Alexey M Kozlov, Diego Darriba, Tomáš Flouri, Benoit Morel, and Alexandros Stamatakis.
509 RAXML-NG: a fast, scalable and user-friendly tool for maximum likelihood phylogenetic
510 inference. *Bioinformatics*, 35(21):4453–4455, 2019.
- 511 12 Benjamin Linard, Krister Swenson, and Fabio Pardi. Rapid alignment-free phylogenetic identi-
512 fication of metagenomic sequences. *Bioinformatics*, 35(18):3303–3312, 01 2019. arXiv:[https://](https://academic.oup.com/bioinformatics/article-pdf/35/18/3303/30024623/btz068.pdf)
513 academic.oup.com/bioinformatics/article-pdf/35/18/3303/30024623/btz068.pdf, doi:
514 10.1093/bioinformatics/btz068.
- 515 13 Kevin Liu, Tandy J Warnow, Mark T Holder, Serita M Nelesen, Jiaye Yu, Alexandros P
516 Stamatakis, and C Randal Linder. SATE-II: very fast and accurate simultaneous estimation of
517 multiple sequence alignments and phylogenetic trees. *Systematic biology*, 61(1):90, 2012.
- 518 14 Frederick A Matsen, Robin B Kodner, and E Virginia Armbrust. pplacer: linear time maximum-
519 likelihood and Bayesian phylogenetic placement of sequences onto a fixed reference tree. *BMC*
520 *bioinformatics*, 11(1):538, 2010.
- 521 15 Frederick A Matsen, Robin B Kodner, and E Virginia Armbrust. pplacer: linear time maximum-
522 likelihood and Bayesian phylogenetic placement of sequences onto a fixed reference tree. *BMC*
523 *bioinformatics*, 11(1):538, 2010.
- 524 16 Connor O McCoy and Frederick A Matsen. Abundance-weighted phylogenetic diversity
525 measures distinguish microbial community states and are robust to sampling depth. *PeerJ*,
526 1:e157, 2013.

- 527 17 Siavash Mirarab. 16S.B.ALL reference tree and alignment. [https://sites.google.com/eng.](https://sites.google.com/eng.ucsd.edu/datasets/alignment/16s23s)
528 [ucsd.edu/datasets/alignment/16s23s](https://sites.google.com/eng.ucsd.edu/datasets/alignment/16s23s).
- 529 18 Siavash Mirarab, Nam Nguyen, Sheng Guo, Li-San Wang, Junhyong Kim, and Tandy Warnow.
530 PASTA: ultra-large multiple sequence alignment for nucleotide and amino-acid sequences.
531 *Journal of Computational Biology*, 22(5):377–386, 2015.
- 532 19 Siavash Mirarab, Nam Nguyen, and Tandy Warnow. SEPP: SATé-enabled phylogenetic
533 placement. In *Biocomputing 2012*, pages 247–258. World Scientific, 2012.
- 534 20 Nam-phuong Nguyen, Siavash Mirarab, Bo Liu, Mihai Pop, and Tandy Warnow. TIPP:
535 taxonomic identification and phylogenetic profiling. *Bioinformatics*, 30(24):3548–3555, 2014.
- 536 21 Morgan N Price, Paramvir S Dehal, and Adam P Arkin. FastTree 2—approximately maximum-
537 likelihood trees for large alignments. *PloS one*, 5(3):e9490, 2010.
- 538 22 Nidhi Shah, Erin K. Molloy, Mihai Pop, and Tandy Warnow. TIPP2: metagenomic taxonomic
539 profiling using phylogenetic markers. *Bioinformatics*, 2021. DOI: 10.1093/bioinformatic-
540 s/btab023.
- 541 23 Vladimir Smirnov and Tandy Warnow. Unblended Disjoint Tree Merging using GTM improves
542 species tree estimation. *BMC genomics*, 21:1–17, 2020.
- 543 24 Alexandros Stamatakis. RAxML-VI-HPC: maximum likelihood-based phylogenetic analyses
544 with thousands of taxa and mixed models. *Bioinformatics*, 22(21):2688–2690, 2006.
- 545 25 J Stoye, D Evers, and F Meyer. Rose: generating sequence families. *Bioinformatics*, 14(2):157–
546 163, 03 1998. arXiv:[https://academic.oup.com/bioinformatics/article-pdf/14/2/157/](https://academic.oup.com/bioinformatics/article-pdf/14/2/157/9731868/140157.pdf)
547 [9731868/140157.pdf](https://academic.oup.com/bioinformatics/article-pdf/14/2/157/9731868/140157.pdf), doi:10.1093/bioinformatics/14.2.157.
- 548 26 Eleanor Wedell, Yirong Cai, and Tandy Warnow. Scalable and accurate phylogenetic
549 placement using pplacer-XR. In *International Conference on Algorithms for Computa-*
550 *tional Biology*, pages 94–105. Springer, 2021. [https://link.springer.com/chapter/10.1007/](https://link.springer.com/chapter/10.1007/978-3-030-74432-8_7)
551 [978-3-030-74432-8_7](https://link.springer.com/chapter/10.1007/978-3-030-74432-8_7).
- 552 27 Eleanor Wedell, Yirong Cai, and Tandy Warnow. SCAMPP: Scaling alignment-based phylo-
553 genetic placement to large trees. *IEEE/ACM Transactions on Computational Biology and*
554 *Bioinformatics*, pages 1–1, 2022. DOI:10.1109/TCBB.2022.3170386.